

[Home](#)

[About](#)

[More](#)

VIDEO PRESENTATION

Click ME!



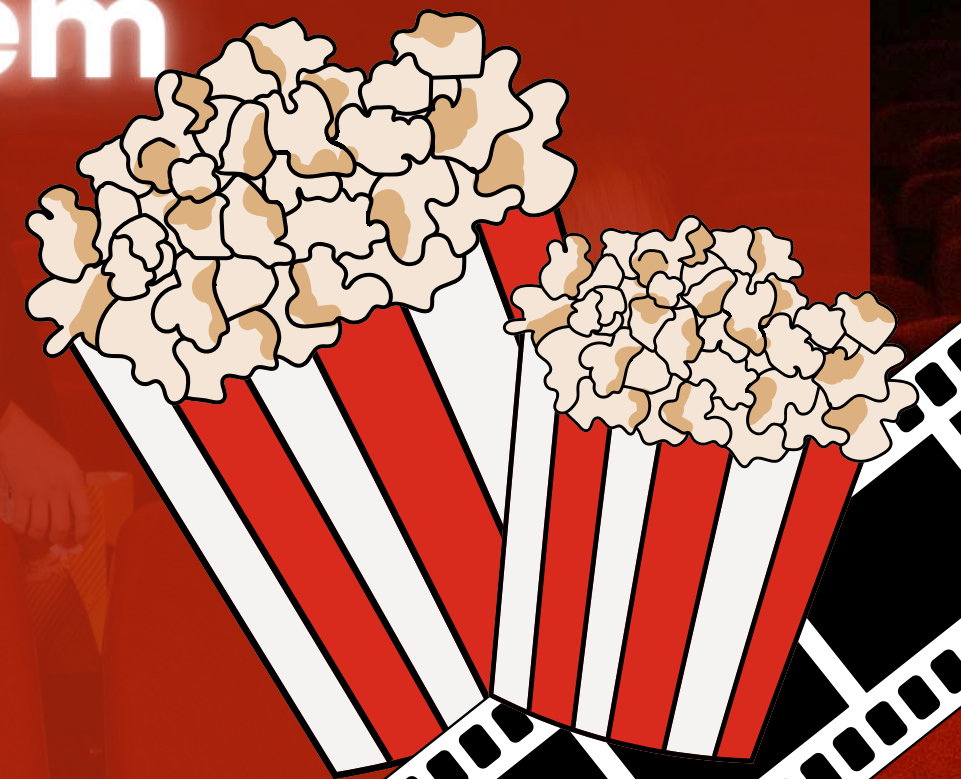
CineTrack

A Smart Movie Tracker System

Lecturer:

TS. DR. Ummi Raba'ah Hashim

START SLIDE



[Home](#)

[About](#)

[More](#)

GROUP MEMBERS

Leader



MUHD UZAIR
B032410498



CHENG XIN HAO
B032410488



CAPURAS JEVEN
B032410987



DHIYA MIRZA
B032410450



DANIEL DEV
B032410485



YOONG LI YANG
B032410439

Home

About

More

PROJECT DESCRIPTION

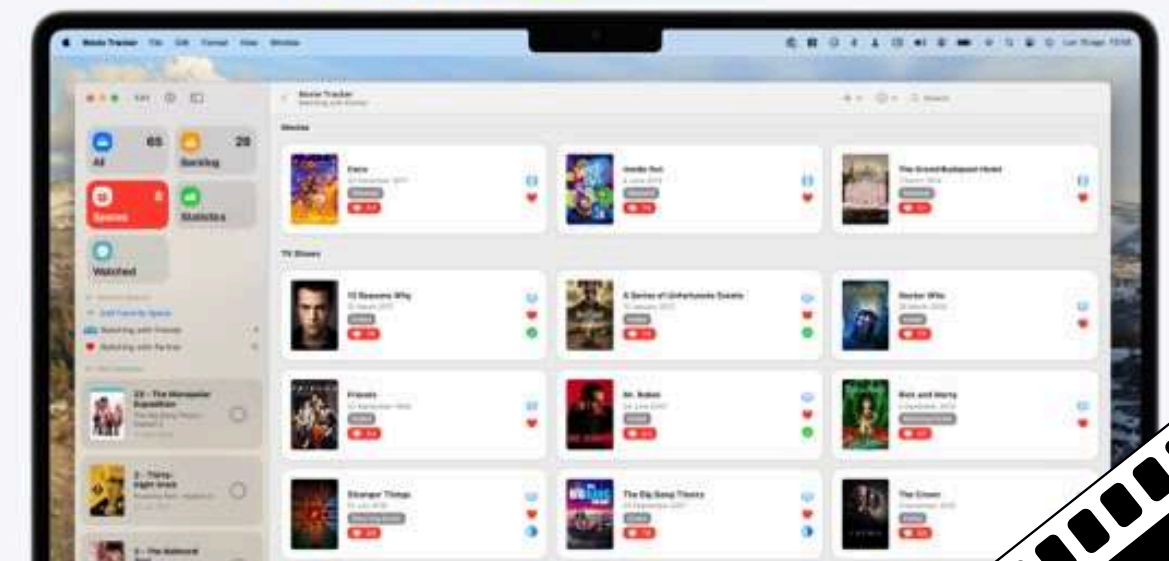
What is CineTrack?

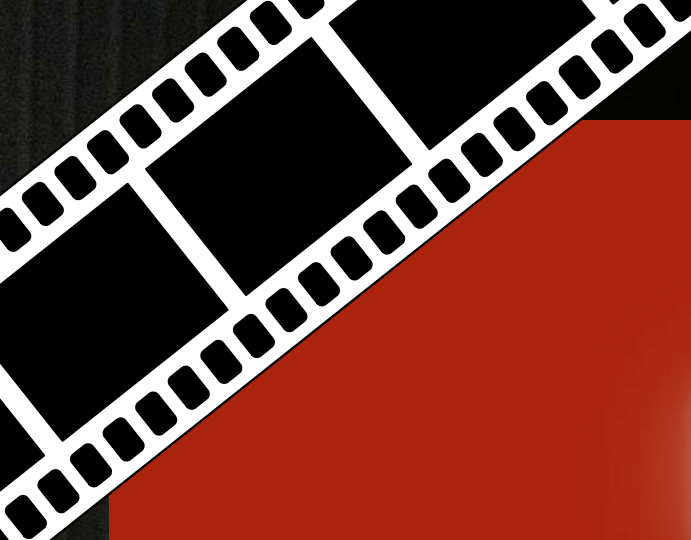
- A COMMAND-LINE MOVIE TRACKER USING C++
- ALLOWS USERS TO ADD, VIEW, SEARCH, AND SORT MOVIES
- STORES DATA DYNAMICALLY USING SINGLY LINKED LIST
- DEMONSTRATES SEARCHING (LINEAR) AND SORTING (BUBBLE)

 **Movie Tracker**



**Track Movies &
TV Shows with easy**





Home

About

More

PROBLEM STATEMENT

📌 No Easy Way to Track Watched Movies

- Users often forget which movies they've watched or plan to watch next. This leads to rewatching unintentionally or missing out on desired titles.

📌 Existing Platforms Are Too Limited or Overcomplicated

- Popular apps like IMDb or Letterboxd either lock features behind accounts, focus on reviews instead of tracking, or overwhelm users with cluttered interfaces.

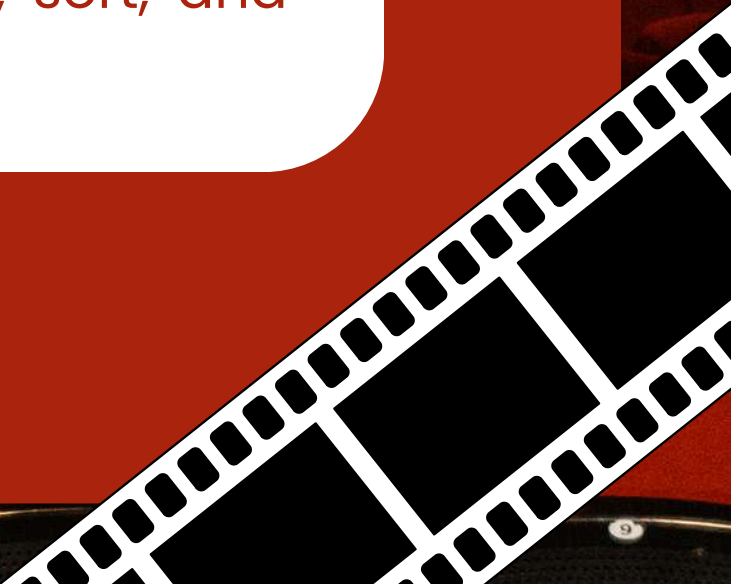
📌 Manual Methods Are Outdated and Error-Prone

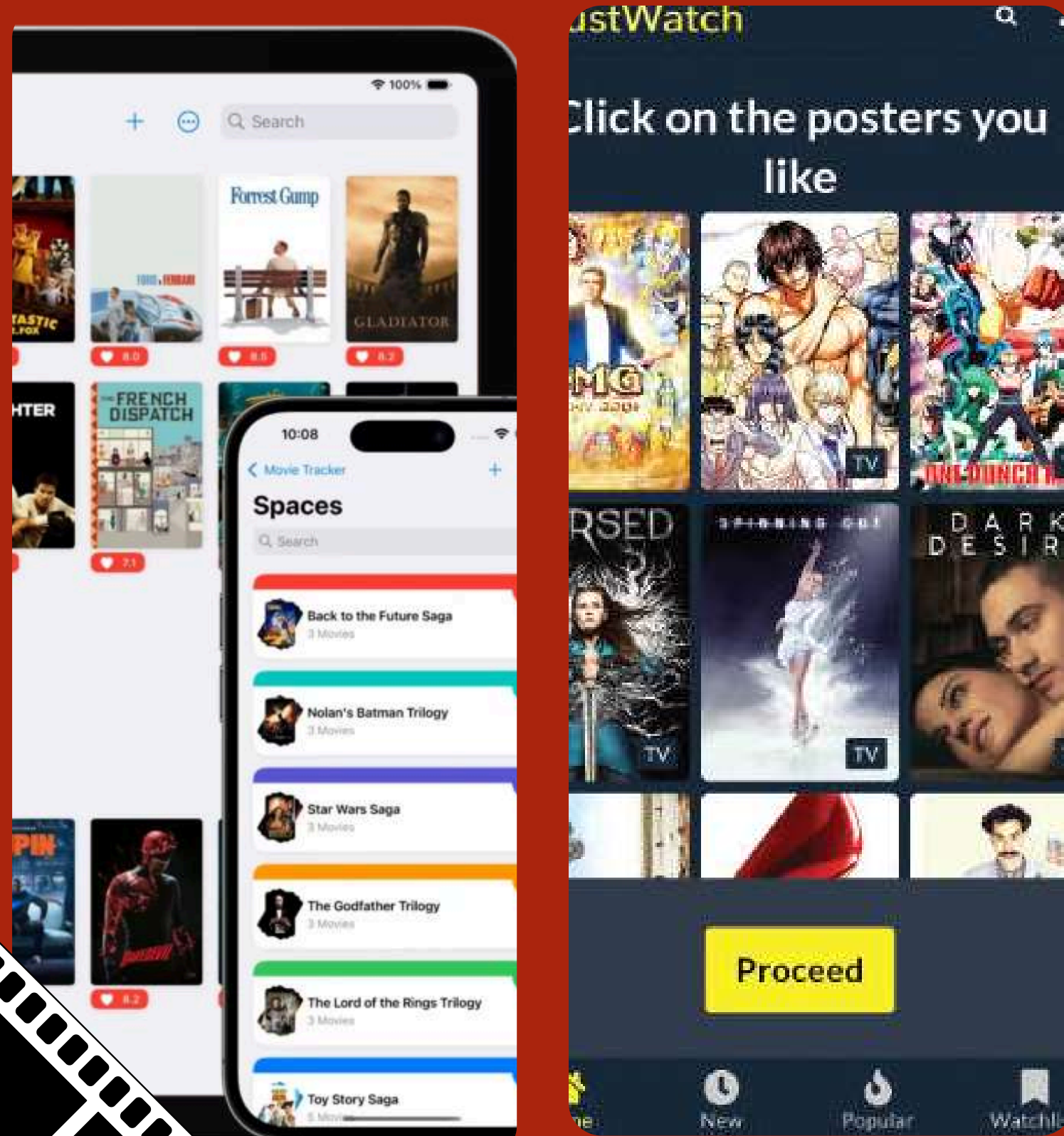
- Using notebooks, spreadsheets, or phone notes is time-consuming, lacks searchability, and doesn't allow sorting by important factors like rating or release year.

📌 Need: A Simple, Offline, Personalized Tracker

- Users need a lightweight console-based program that allows them to easily store, view, sort, and search their movie collection, all offline, without registration or distractions.

This system addresses the real-world need for movie enthusiasts to easily keep track of their watchlist and organize movies based on personal preferences, all through a clean, user-friendly console interface.





KEY FEATURES

- ADD A MOVIE (TITLE, GENRE, RATING, YEAR)
- VIEW ALL STORED MOVIES
- SEARCH MOVIE BY TITLE USING LINEAR SEARCH
- SORT BY RATING OR RELEASE YEAR USING BUBBLE SORT
- CHOOSE ASCENDING OR DESCENDING ORDER

Why CineTrack?

- Useful for movie lovers to manage watched/to-watch lists
- Prioritize best-rated films using sorting
- Fast title look-up using search
- Could evolve into a full-featured media tracker

DATA STRUCTURE & ALGORITHM USED

**LINEAR
SEARCH**

Algorithm

**BUBBLE
SORT**

Algortihm

LINKED LIST

Data Stucture

LINEAR SEARCH – STEP BY STEP

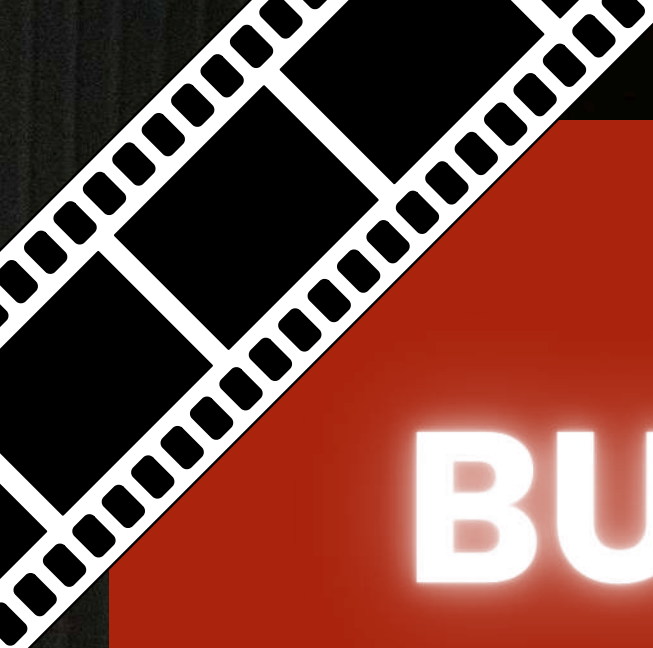
USED IN: MOVIELIST::SEARCH() FUNCTION

PURPOSE:

Search for a movie by its title in a linked list.

ALGORITHM STEPS:

1. Get target movie title from user input
2. Start from the head of the linked list
3. Compare each node's title with the target
4. If match is found, then display movie details
5. If no match found till the end of the list, then display 'not found'



[Home](#)

[About](#)

[More](#)

BUBBLE SORT – STEP BY STEP

USED IN: MOVIELIST::SORT() FUNCTION

PURPOSE:

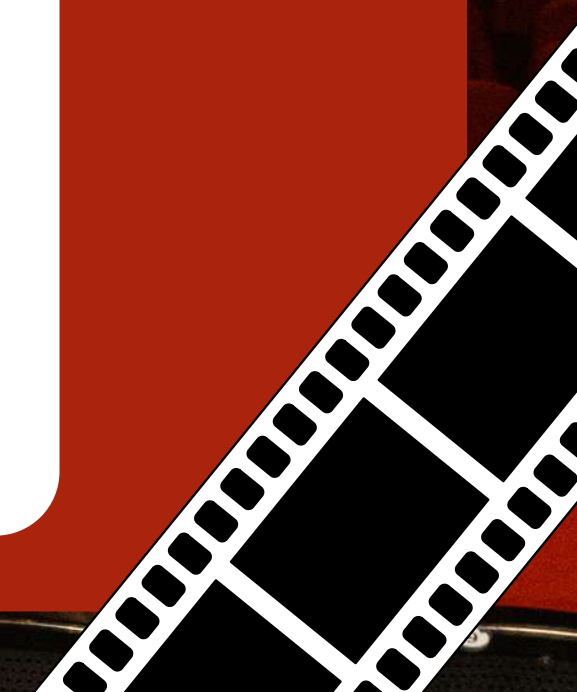
Sort movies based on:

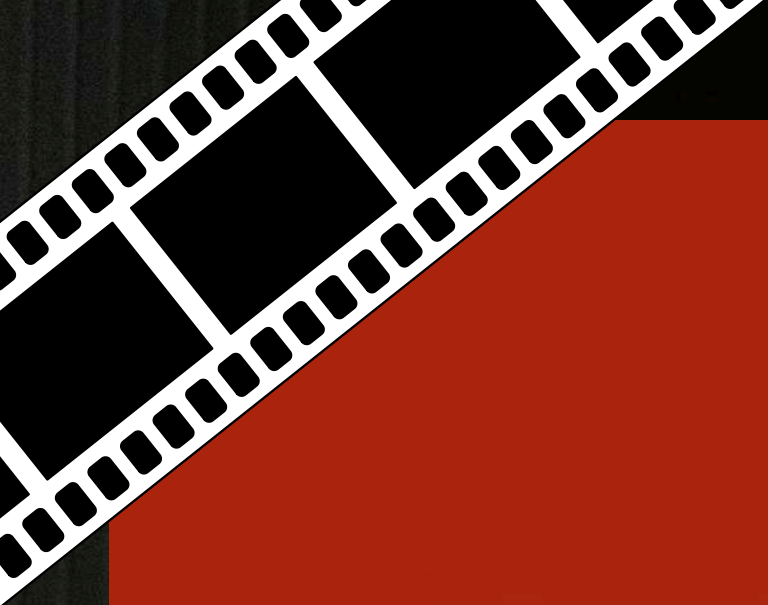
- Rating OR
- Year of Release

SORTING OPTIONS:

- Ascending (Low → High)
- Descending (High → Low)

ALGORITHM STEPS:

1. Loops through the linked list using two pointers
 2. Compares movie data fields (rating or year)
 3. Swaps only the node data, not the pointers (via swap() function)
 4. Repeats until no more swaps are needed (swapp == 0)
- 



[Home](#)

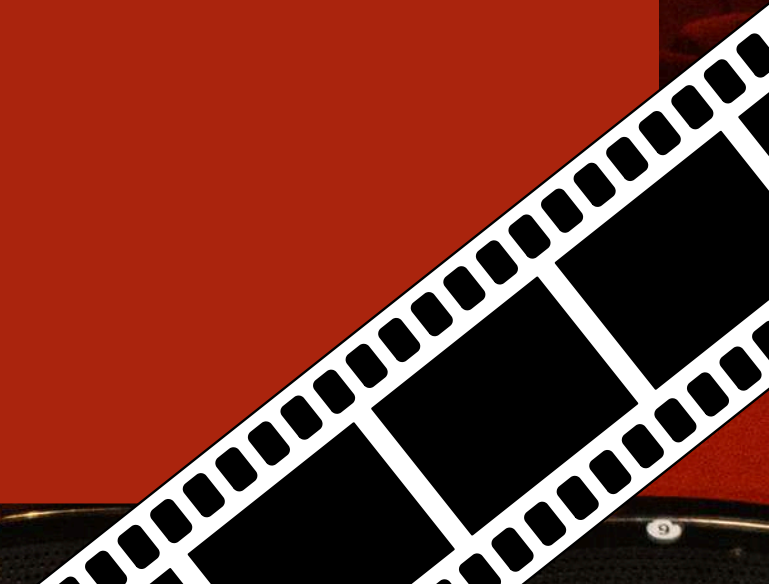
[About](#)

[More](#)

CODE OVERVIEW

----- Welcome to CineTrack! -----

● PROGRAM WELCOME MESSAGE



CODE OVERVIEW : ADDING, DISPLAYING, SEARCHING

```
42  
43 void MovieList::add(string t, string g, float r, int y)  
44 {  
45     Node* pNew = new Node;  
46     pNew->title = t;  
47     pNew->genre = g;  
48     pNew->rating = r;  
49     pNew->year = y;  
50     pNew->link = head;  
51     head = pNew;  
52     count = count + 1;  
53 }
```

● CODE SNIPPET (ADDING):

- In add() function, will add new movie at head of list
- Efficient insertion

```
while ((pCurrent->title != target) && (pCurrent->link != 0))  
{  
    pCurrent = pCurrent->link;  
}  
if (pCurrent->title == target)  
{  
    cout << "\nFound a movie with '" << target << "' genre" << endl << endl;  
    cout << "Title: " << pCurrent->title << endl;  
    cout << "Genre: " << pCurrent->genre << endl;  
    cout << "Rating: " << pCurrent->rating << endl;  
    cout << "Year Released: " << pCurrent->year << endl<<endl;  
}  
else if (pCurrent->title != target)  
{  
    cout << "No available movie for '" << target << "' genre\n" << endl;
```

● LINEAR SEARCH IN THE CODE

1. Get target movie title from user input
2. Start from the head of the linked list
3. Compare each node's title with the target
4. If match is found, then display movie details
5. If no match found till the end of the list, then display 'not found'

```
27  
28 else  
29 {  
30     cout << " ---- LIST OF ALL MOVIES ---- \n\n";  
31     for (int start = 0; start != count; start++)  
32     {  
33         cout << "Movie " << start + 1 << endl;  
34         cout << "Title: " << pCurrent->title << endl;  
35         cout << "Genre: " << pCurrent->genre << endl;  
36         cout << "Rating: " << pCurrent->rating << endl;  
37         cout << "Year Released: " << pCurrent->year << endl;  
38         pCurrent = pCurrent->link;  
39     }
```

● CODE SNIPPET (DISPLAYING):

- In display() function, will loops through and prints all nodes

[Home](#)[About](#)[More](#)

CODE OVERVIEW: SORTING LOGIC

● BUBBLE SORT IN THE CODE

1. Loops through the linked list using two pointers
2. Compares movie data fields (rating or year)
3. Swaps only the node data, not the pointers (via swap() function)
4. Repeats until no more swaps are needed (swapp == 0)

```
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
case 1:
    if (order == 'a' || order == 'A')
    {
        if (ptr1->rating > ptr1->link->rating)
        {
            swap(ptr1, ptr1->link);
            swapp = 1;
        }
    }
    else if (order == 'd' || order == 'D')
    {
        if (ptr1->rating < ptr1->link->rating)
        {
            swap(ptr1, ptr1->link);
            swapp = 1;
        }
    }
    break;
case 2:
    if (order == 'a' || order == 'A')
    {
        if (ptr1->year > ptr1->link->year)
```



CODE SNIPPET OF SORTING

DEMO - SCREENSHOTS / FLOW

```
----- Welcome to CineTrack! -----  
- Track all your favourite shows here!-
```

```
--- MENU ---  
1 - Add a Movie  
2 - Display All Movies  
3 - Search Movie by Title  
4 - Sort Movie  
5 - Exit Menu
```

Choice: |

● WELCOME PAGE

```
----- FIND A MOVIE -----
```

Enter the movie name: Kpop Demon Hunter

Found a movie with 'Kpop Demon Hunter' name

Title: Kpop Demon Hunter

Genre: Music

Rating: 10

Year Released: 2025

● SEARCHING A MOVIE

```
----- ADD A MOVIE -----
```

Title: Abang Long Faiz

Genre: Action

Rating: 7

Released Year: 2028|

● ADDING A MOVIE

----- LIST OF ALL MOVIES -----

Movie 1
Title: Kpop Demon Hunter
Genre: Music
Rating: 10
Year Released: 2025

Movie 2
Title: Boboiboy Movie 7
Genre: Kids
Rating: 9
Year Released: 2059

Movie 3
Title: Abang Long Faiz
Genre: Action
Rating: 7
Year Released: 2028

● DISPLAYING ALL MOVIES

DEMO - SCREENSHOTS / FLOW

----- FIND A MOVIE -----

Enter the movie name: Kpop Demon Hunter

Found a movie with 'Kpop Demon Hunter' name

Title: Kpop Demon Hunter
Genre: Music
Rating: 10
Year Released: 2025

● SEARCHING A MOVIE

----- SORT MOVIE -----

--- Choose Sorting ---
1 - Sort by Rating
2 - Sort by Year Released

Choice: 1
Ascending [a] or Descending [d]? a

----- LIST OF ALL MOVIES -----

Movie 1
Title: Abang Long Faiz
Genre: Action
Rating: 7
Year Released: 2028

Movie 2
Title: Boboiboy Movie 7
Genre: Kids
Rating: 9
Year Released: 2059

Movie 3
Title: Kpop Demon Hunter
Genre: Music
Rating: 10
Year Released: 2025

● SORTING A MOVIE
BY RATING IN ASC.


```
--- Choose Sorting ---  
1 - Sort by Rating  
2 - Sort by Year Released  
  
Choice: 2  
Ascending [a] or Descending [d]? d
```

```
----- LIST OF ALL MOVIES -----
```

```
Movie 1  
Title: Boboiboy Movie 7  
Genre: Kids  
Rating: 9  
Year Released: 2059
```

```
Movie 2  
Title: Abang Long Faiz  
Genre: Action  
Rating: 7  
Year Released: 2028
```

```
Movie 3  
Title: Kpop Demon Hunter  
Genre: Music  
Rating: 10  
Year Released: 2025
```

● SORT MOVIES, BY YEAR IN DESC.

```
----- SORT MOVIE -----
```

```
--- Choose Sorting ---  
1 - Sort by Rating  
2 - Sort by Year Released
```

```
Choice: 3  
Ascending [a] or Descending [d]? d  
Press any key to continue
```

● PROGRAM QUIT IF WRONG INPUT

```
Choice: 6
```

```
Invalid choice! Try again.  
Press any key to continue . . . |
```

● PROGRAM QUIT IF WRONG INPUT

DEMO -
SCREENSHOTS /
FLOW


```
--- MENU ---  
1 - Add a Movie  
2 - Display All Movies  
3 - Search Movie by Title  
4 - Sort Movie  
5 - Exit Menu
```

Choice:

[Home](#)

[About](#)

[More](#)

LIVE DEMONSTRATION



[Click ME!](#)



[Home](#)

[About](#)

[More](#)

END OF DEMONSTRATION

----- See you again on CineTrack! -----

● PROGRAM FAREWELL MESSAGE

CHALLENGES & SOLUTIONS

No	Challenges	Solutions
1	Handling multi-word movie titles (e.g., "The Batman")	Replaced cin with getline() to read full lines of input
2	Difficulty swapping nodes in a singly linked list	Simplified sorting by swapping node data instead of manipulating pointers
3	Program crashes or incorrect behavior on empty list	Added checks to ensure the list is not empty before searching or sorting
4	Sorting logic didn't cover all edge cases (e.g., last node)	Carefully adjusted the loop logic and used a tail pointer (lptr) to optimize passes
5	Hard to test search/sort output without visual feedback	Displayed the list automatically after sorting and provided clear output messages
6	Input buffer issues (e.g., skipped inputs)	Used cin.ignore() to clear leftover newline characters before getline()

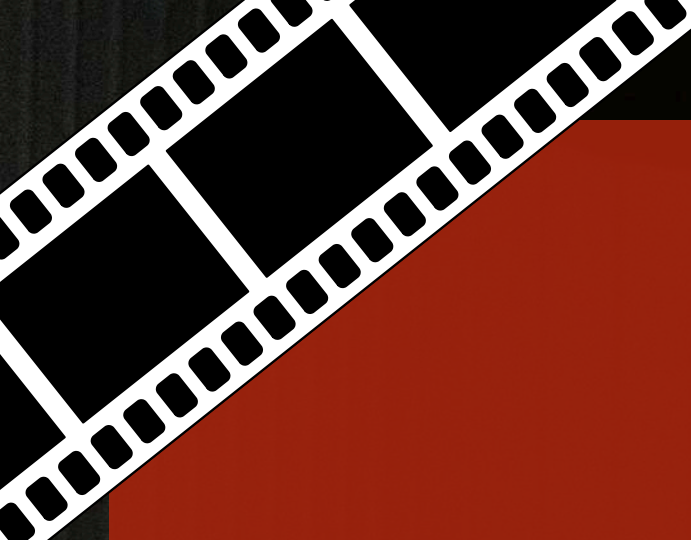
[Home](#)[About](#)[More](#)

CONCLUSION

CineTrack shows how basic programming techniques can solve real-life problems. Our program helps users manage their movie collection by **adding, viewing, searching, and sorting movies**. We used a **linked list to store data, linear search to find movies by title, and bubble sort to sort them by rating or year**. CineTrack is simple, offline, and easy to use. In the future, we can improve it by adding features like editing, saving to a file, or a GUI. This project proves that even basic algorithms can create useful and practical applications.

THE END





[Home](#)

[About](#)

[More](#)

2 0 2 5 C I N E M A P R E S E N T A T I O N

THANK YOU

SEE YOU NEXT TIME

